

HYBRID Cryptographic Verification Protocol Specification

Document Ref:	SPEC-CRYPTO-PQC-2026-V1	Compliance Standard:	NIST FIPS 186-4 / FIPS 204
Classification:	Public / Audit Deliverable	Implementation:	Offline / WASM / Native Client

1. OVERVIEW & SYSTEM OBJECTIVES

This specification defines the cryptographic architecture for verifying digital signatures using a **Dual-Mechanism Hybrid Post-Quantum Signature Scheme**. The system mitigates both current classical attack vectors and future quantum computing threats (e.g., Shor's algorithm running on a Cryptanalytically Relevant Quantum Computer) by executing concurrent verification across two independent cryptographic families:

- **Classical Elliptic Curve Cryptography (ECC):** ECDSA using the `secp256r1` (P-256) curve.
- **Post-Quantum Lattice-Based Cryptography:** Crystals-Dilithium Level 3 / ML-DSA-65.

Under this architecture, a verification attempt outputs **TRUE (PASS)** if and only if both independent cryptographic primitives evaluate to valid signatures over the exact payload digest.

2. GOVERNING STANDARDS & CRYPTOGRAPHIC PRIMITIVES

Functional Layer	Algorithm / Scheme	Parameter / Curve Set	Governing Standard	Quantum Resistance
Message Digest	SHA-256	256-bit Digest	NIST FIPS 180-4	Pre-image (Grover) ~128-bit
Classical Layer	ECDSA	Curve <code>secp256r1</code> (P-256)	NIST FIPS 186-4 / SEC 1	None (Vulnerable to Shor's)
Post-Quantum Layer	Crystals-Dilithium	Level 3 / ML-DSA-65	NIST FIPS 204	Category 3 (~192-bit security)
Envelope Transport	Concatenated Binary	Deterministic Slicing	Internal Specification	N/A (Deterministic Framing)

3. ENVELOPE & PAYLOAD SPECIFICATIONS

To eliminate parsing vulnerabilities, DER ambiguity, and ASN.1 malleability attacks (such as BIP-66 compliance issues), public keys and signatures are transmitted as raw fixed-offset byte arrays encoded in Base64.

3.1 Composite Hybrid Public Key Structure (2,017 Bytes Total)

Offset 0 .. 64 (65 Bytes)	Offset 65 .. 2016 (1,952 Bytes)
ECDSA Public Key (SEC 1)	Dilithium3 / ML-DSA-65 Public Key

- **ECDSA Public Key (65 Bytes):** Encoded in SEC 1 uncompressed format (`0x04` prefix byte || 32-byte *X*-coordinate || 32-byte *Y*-coordinate).
- **ML-DSA-65 Public Key (1,952 Bytes):** Encoded per NIST FIPS 204 polynomial matrix specification.

3.2 Composite Hybrid Signature Structure (3,357 Bytes Total)

Offset 0 .. 63 (64 Bytes)	Offset 64 .. 3356 (3,293 Bytes)
ECDSA Signature (<i>R</i> <i>S</i>)	Dilithium3 / ML-DSA-65 Signature

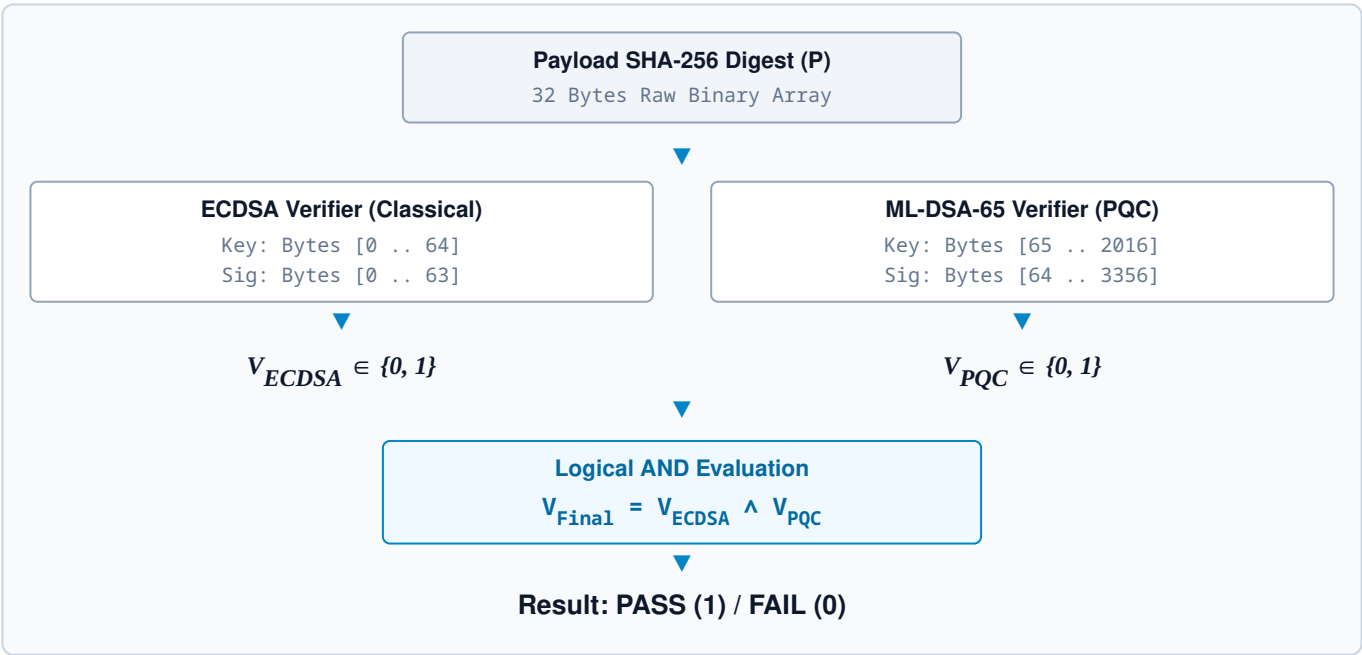
- **ECDSA Signature (64 Bytes):** Fixed-length big-endian concatenation of 32 bytes *R* || 32 bytes *S*.
- **ML-DSA-65 Signature (3,293 Bytes):** Encoded raw polynomial vectors (*c̃*, *z*, *h*).

4. FORMAL VERIFICATION PROTOCOL & LOGIC FLOW

4.1 Inputs

- *H*: A 64-character Hexadecimal string representing the pre-computed SHA-256 payload digest.
- σ_{Hybrid} : Base64 encoded binary string of length 3,357 bytes.
- K_{Hybrid} : Base64 encoded binary string of length 2,017 bytes.

4.2 Execution Logic Flow



4.3 Algorithm Execution Steps

1. **Payload Extraction:** Convert H to raw 32-byte binary array $P = \text{HexDecode}(H)$.

2. **Key Slicing:**

$$\begin{aligned} K_{ECDSA} &= K_{Hybrid}[0 \dots 64] \\ K_{PQC} &= K_{Hybrid}[65 \dots 2016] \end{aligned}$$

3. **Signature Slicing:**

$$\begin{aligned} \sigma_{ECDSA} &= \sigma_{Hybrid}[0 \dots 63] \\ \sigma_{PQC} &= \sigma_{Hybrid}[64 \dots 3356] \end{aligned}$$

4. **Classical Evaluation:**

- Extract EC point $Q = (X, Y)$ from K_{ECDSA} and confirm $Q \in E(F_p)$ for `secp256r1`.
- Extract integers $r = \sigma_{ECDSA}[0 \dots 31]$ and $s = \sigma_{ECDSA}[32 \dots 63]$.
- Compute $V_{ECDSA} = \text{Verify}_{ECDSA}(Q, P, r, s)$.

5. **Post-Quantum Evaluation:**

- Reconstruct matrix A and polynomial vectors from K_{PQC} .
- Compute $V_{PQC} = \text{Verify}_{ML-DSA-65}(K_{PQC}, P, \sigma_{PQC})$.

6. **Assertion:**

$$V_{Final} = V_{ECDSA} \wedge V_{PQC}$$

5. SECURITY & THREAT ANALYSIS

5.1 Defense in Depth (Dual Security Assumption)

The hybrid architecture operates under an "OR" threat model vulnerability and an "AND" security requirement:

- **Scenario A (Quantum Computer Emergence):** If a quantum adversary breaks `secp256r1`, V_{ECDSA} may be forged, but V_{PQC} remains unbroken due to the Module Learning With Errors (MLWE) hard lattice problem.
- **Scenario B (Mathematical Breakthrough in Lattices):** If an unexpected classical vulnerability is discovered in lattice cryptography, V_{PQC} may be bypassed, but V_{ECDSA} remains secured by the Discrete Logarithm Problem (DLP).

5.2 Malleability Immunity

By utilizing deterministic fixed-offset byte slicing for $R \parallel S$ rather than variable-length ASN.1 DER sequence structures, signature canonicalization issues are completely eliminated.

6. AUDITOR INDEPENDENT VERIFICATION INSTRUCTIONS

Method A: Primary Verification (Packaged Auditor Kit)

1. Extract `Offline_Hybrid_Signature_Auditor_Kit.zip`.
2. Open `index.html` in an isolated / air-gapped web browser.
3. Supply target inputs: SHA-256 Hash (32-byte hex), Hybrid Signature (Base64, 3,357 bytes), Hybrid Public Key (Base64, 2,017 bytes).
4. Verify that both **Step 1 (ECDSA)** and **Step 2 (Dilithium Level 3)** log `PASSED` status in execution log.

Method B: Manual Fallback Build Procedure (Dart CLI)

If `Offline_Hybrid_Signature_Auditor_Kit.zip` is not available, construct the auditor kit using the procedure below:

1. Install Dart SDK in Windows PowerShell:

```
winget install --id Google.DartSDK -e
```

(Close PowerShell completely and open a fresh terminal window so Windows picks up the updated system paths).

2. Create Directory & File Structure:

Create a directory named `hybrid_auditor_kit` with the following layout:

```
hybrid_auditor_kit/  
├─ pubspec.yaml  
└─ web/  
    ├─ index.html  
    └─ main.dart
```

3. Fetch Cryptographic Dependencies:

```
dart pub get
```

4. Compile Dart to Web JavaScript:

```
dart compile js web/main.dart -o web/main.js
```

5. Execute Verification:

Confirm that `web/main.js` was successfully created. Double-click `web/index.html` in Windows File Explorer and input the required cryptographic payload and keys to run offline verification.